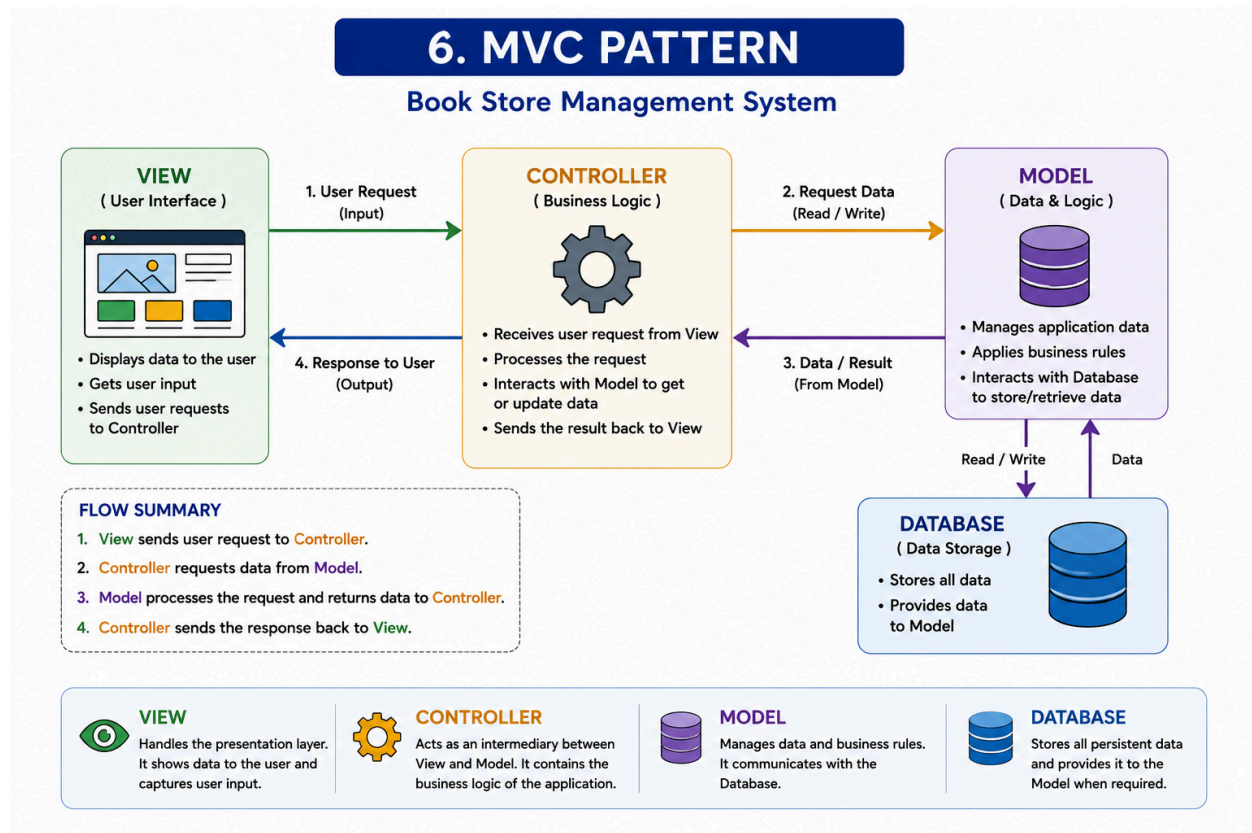


MVC PATTERN

The **Book Store backend application** follows the **Model-View-Controller (MVC)** architectural pattern. MVC is a software design approach that separates an application into three interconnected layers: **Model, View, and Controller**. This separation improves code organization, modularity, maintainability, and scalability.



1. Model Layer (Data Layer)

The **Model layer** is responsible for handling all data-related logic in the Book Store application. It defines the structure of the data and performs operations related to the database. In this project, the models are implemented using **Mongoose**, which provides a schema-based solution for modeling application data in **MongoDB**.

The Model layer includes schemas related to:

- User
- Book
- Cart
- Order

- Seller
- Admin

The Model layer is responsible for:

- Defining database schemas
- Performing CRUD operations
- Managing relationships between entities
- Interacting directly with MongoDB

2. Controller Layer

The **Controller layer** acts as an intermediary between the **View (Routes)** and the **Model**. It receives incoming requests from the client, processes the request data, performs validation if required, calls the appropriate model methods, and sends the response back to the user.

Examples of controller responsibilities in the Book Store project include:

- Handling user registration and login
- Fetching available books
- Processing book search and category selection
- Adding books to the cart
- Placing orders
- Retrieving order history
- Managing seller and admin operations

3. View Layer (Routing Layer)

In the backend of the Book Store project, the **View layer** is represented by the **routing layer**. The routes define different API endpoints and determine how the backend responds to HTTP requests such as **GET**, **POST**, **PUT**, and **DELETE**.

The routing layer is responsible for:

- Defining API endpoints
- Mapping requests to the correct controller functions
- Handling communication between client requests and server logic

Examples of routes in the Book Store project include:

- User routes
- Book routes
- Cart routes
- Order routes

- Admin/Seller routes

4. Database

The **Database** is the storage layer of the Book Store application and is connected to the Model layer. It stores all important data such as:

- User account information
- Book details
- Cart items
- Orders and order history
- Seller and admin details

The Model performs database operations such as inserting, updating, deleting, and retrieving data from MongoDB.

5. MVC Flow in the Book Store Project

The flow of the MVC architecture in the Book Store application is as follows:

1. The **User** sends a request through the frontend.
2. The request is received by the **View layer (Routes)**.
3. The route forwards the request to the appropriate **Controller**.
4. The **Controller** processes the request and interacts with the **Model**.
5. The **Model** performs the required database operations with MongoDB.
6. The result is returned to the **Controller**.
7. The **Controller** sends the response back to the user.

Advantages of Using MVC in This Project

- **Separation of Concerns:** Each layer has a clearly defined responsibility, improving readability and maintainability.
- **Scalability:** New features can be added easily by creating new routes, controllers, and models.
- **Reusability:** Logic in controllers and models can be reused across multiple parts of the application.
- **Testing:** Each layer can be tested independently, especially the models and controllers.
- **Collaboration-Friendly:** Multiple developers can work on different layers of the application simultaneously.